



# Laboratorio di Elettronica Digitale

## - Sequential Logic Circuits -



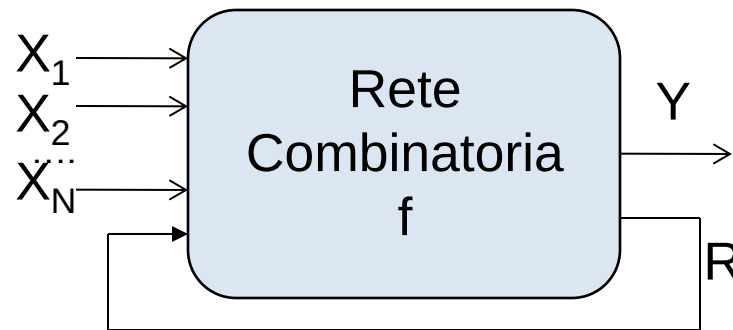
## - Reti Sequenziali -

- Nelle reti sequenziali le uscite  $Y$  dipendono dallo *stato*  $S$  e dagli ingressi  $I$

$$Y = f(S, I)$$

- Lo stato rappresenta a sua volta la storia della rete
- Si distinguono in:
  - Reti sequenziale asincrone
    - La variazione di un ingresso può immediatamente provocare una transizione di stato
  - Reti sequenziali sincrone
    - Le transizioni di stato avvengono solo in momenti definiti (tipicamente scanditi dal fronte del clock)

## - Reti Sequenziali Asincrone -



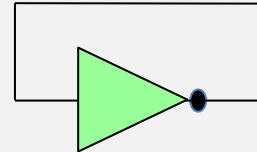
- Lo stato è rappresentato da una condizione “stabile” della rete
- La presenza di condizioni stabili rappresenta di fatto “una memoria” o stato
- La variazione degli ingressi provoca il passaggio verso “condizioni di stabilità” distinte
- Per esempio: flip-flop SR, JK, D, latch

## - Reti Sequenziali Asincrone -

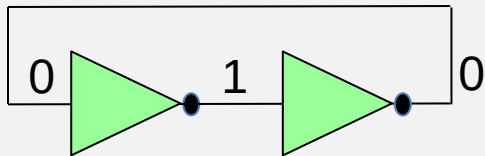
Examples :

This circuit **has no stable state**:

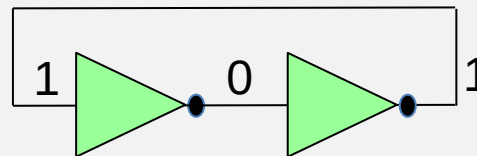
The circuit will oscillate between 0-1.



This circuit has **2 stable states**: it represents a 1-bit memory. However,

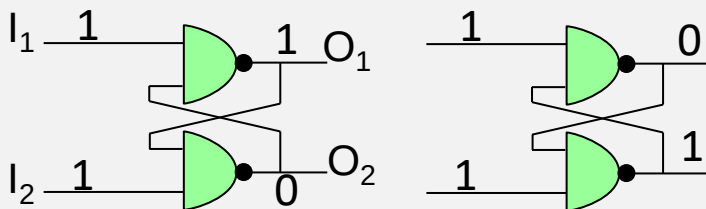


and



there is no easy way to switch between the states. (no input)

The **latch** has 2 stable states **with the possibility to switch between them**.



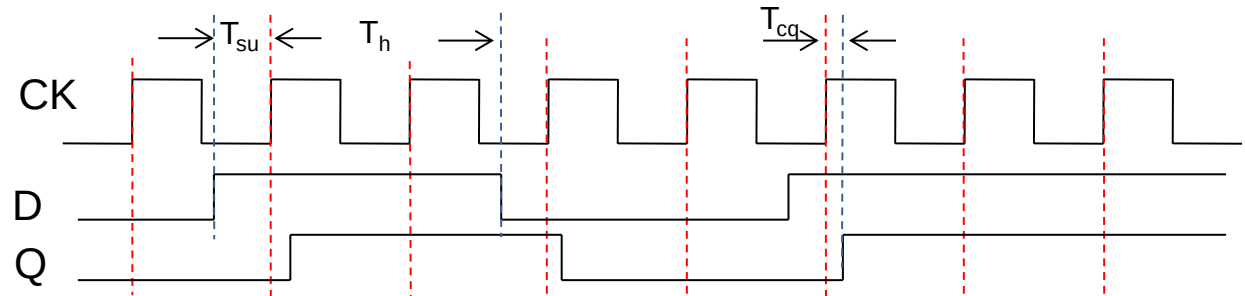
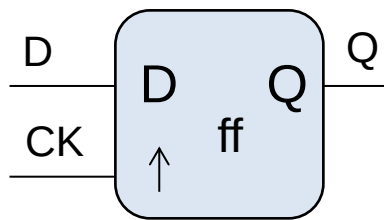
| $I_1$ | $I_2$ | $O_1$ | $O_2$ |
|-------|-------|-------|-------|
| 1     | 1     | $O_1$ | $O_2$ |
| 0     | 1     | 1     | 0     |
| 1     | 0     | 0     | 1     |
| 0     | 0     | -     | -     |

A '0' input puts the corresponding output to 1. Both inputs to 1 hold the current state.

The latch is a «writable» 1-bit memory

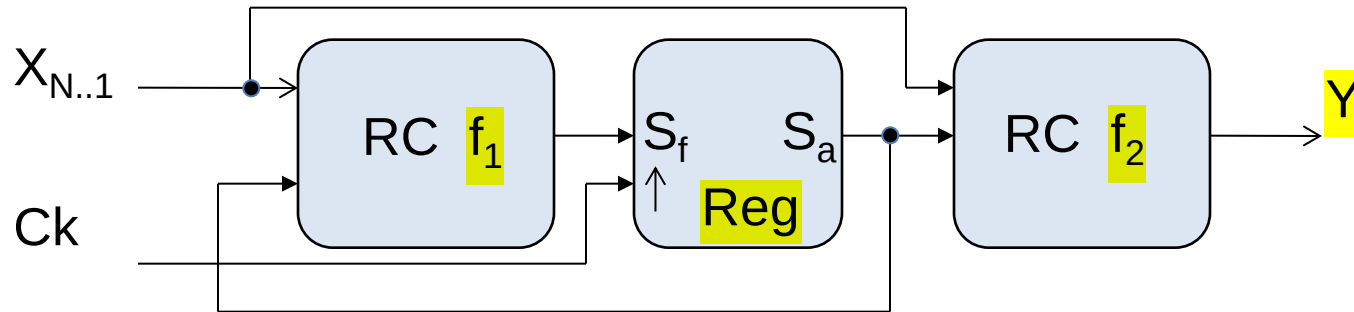
## - Synchronous Circuits -

- In a asynchronous sequential circuit the state changes immediately when an input changes
- In a synchronous (sequential) circuits the state changes occur only at predetermined moments (typically at the rising edge of the clock signal)
- The “heart” of every synchronous circuit is the flip-flop D, or register.



- The ff samples the input D at the rising edge of the clock. In real circuits the set-up ( $T_{su}$ ), hold ( $T_h$ ) and clock-to-q ( $T_{cq}$ ) **constraints should be applied** (see Elettronica dei Sistemi Digitali course)

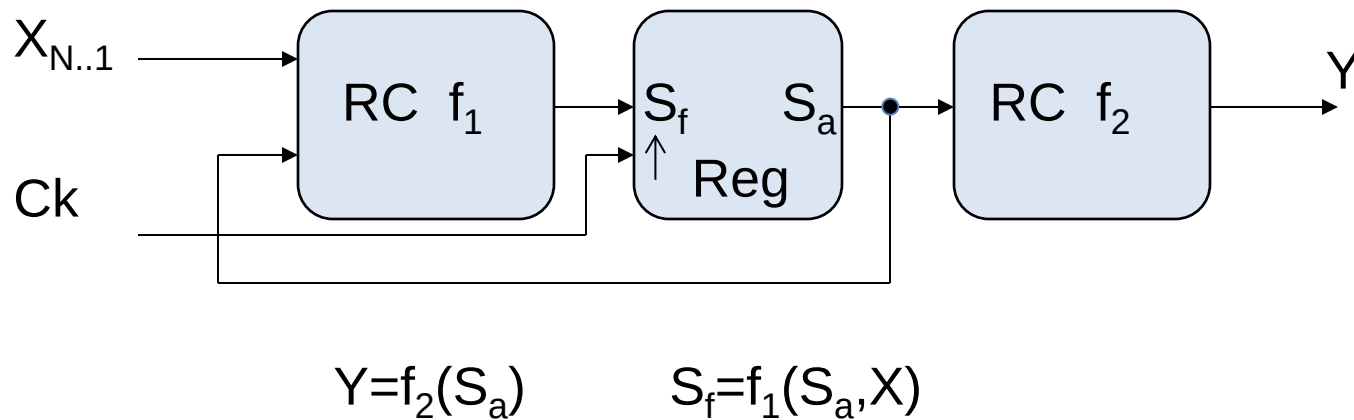
## - Finite state machine -



$$Y = f_2(S_a, X) \quad S_f = f_1(S_a, X)$$

- Le transizioni di stato ( $S_f \rightarrow S_a$ ) avvengono sul fronte attivo del CK
- Lo stato è tipicamente “memorizzato” in un registro
- Nel caso in cui le uscite dipendano direttamente dagli ingressi, la macchina è del tipo “Mealy”

## - Finite state machine -



- Nel caso in cui le uscite dipendano SOLO dallo stato, la macchina è del tipo “Moore”

## - Finite state machine -

### - Esempio: Contatore up a 4 bit

➤ Si devono sintetizzare le funzioni  $f_1$  e  $f_2$

| Funzione $f_1$      |                    |
|---------------------|--------------------|
| Stato attuale $S_a$ | Stato Futuro $S_f$ |
| $I_3 I_2 I_1 I_0$   | $Q_3 Q_2 Q_1 Q_0$  |
| 0000                | 0001               |
| 0001                | 0010               |
| 0010                | 0011               |
| 0011                | 0100               |
| 0100                | 0101               |
| 0101                | 0110               |
| 0110                | 0111               |
| 0111                | 1000               |
| 1000                | 1001               |
| 1001                | 1010               |
| 1010                | 1011               |
| 1011                | 1100               |
| 1100                | 1101               |
| 1101                | 1110               |
| 1110                | 1111               |
| 1111                | 0000               |

➤  $f_1$  è una funzione 4 ingressi/4 uscite

$$Q_0 = \overline{I_0}$$

$$Q_1 = \overline{I_1} I_0 + I_1 \overline{I_0}$$

$$Q_2 = \overline{I_2} I_1 I_0 + I_2 \overline{I_1} \overline{I_0} + I_2 \overline{I_1} I_0 + I_2 I_1 \overline{I_0}$$

$$Q_3 = \overline{I_3} I_2 I_1 I_0 + I_3 \overline{I_2} \overline{I_1} \overline{I_0} + I_3 \overline{I_2} \overline{I_1} I_0 + I_3 \overline{I_2} I_1 \overline{I_0} +$$

$$+ I_3 \overline{I_2} I_1 I_0 + I_3 I_2 \overline{I_1} \overline{I_0} + I_3 I_2 \overline{I_1} I_0 + I_3 I_2 I_1 \overline{I_0}$$



## - Reti Sequenziali Sincrone -

### - Esempio: Contatore up a 4 bit

➤ Si devono sintetizzare le funzioni  $f_1$  e  $f_2$

| Funzione $f_2$ |                |
|----------------|----------------|
| Ingresso       | Uscita         |
| $A_3A_2A_1A_0$ | $B_3B_2B_1B_0$ |
| 0000           | 0000           |
| 0001           | 0001           |
| 0010           | 0010           |
| 0011           | 0011           |
| 0100           | 0100           |
| 0101           | 0101           |
| 0110           | 0110           |
| 0111           | 0111           |
| 1000           | 1000           |
| 1001           | 1001           |
| 1010           | 1010           |
| 1011           | 1011           |
| 1100           | 1100           |
| 1101           | 1101           |
| 1110           | 1110           |
| 1111           | 1111           |

➤  $f_2$  la funzione identità

$$B_0 = A_0$$

$$B_1 = A_1$$

$$B_2 = A_2$$

$$B_3 = A_3$$

## - Reti Sequenziali Sincrone -

### - Esempio: Contatore up a 3 bit con reset

➤ Si devono sintetizzare le funzioni  $f_1$  e  $f_2$

| Funzione $f_1$   |               |                 |
|------------------|---------------|-----------------|
| Stato attuale Sa |               | Stato Futuro Sf |
| R                | $I_2 I_1 I_0$ | $Q_2 Q_1 Q_0$   |
| 0                | 000           | 001             |
| 0                | 001           | 010             |
| 0                | 010           | 011             |
| 0                | 011           | 100             |
| 0                | 100           | 101             |
| 0                | 101           | 110             |
| 0                | 110           | 111             |
| 0                | 111           | 000             |
| 1                | 000           | 000             |
| 1                | 001           | 000             |
| 1                | 010           | 000             |
| 1                | 011           | 000             |
| 1                | 100           | 000             |
| 1                | 101           | 000             |
| 1                | 110           | 000             |
| 1                | 111           | 000             |

➤  $f_1$  è una funzione 4 ingressi/3 uscite

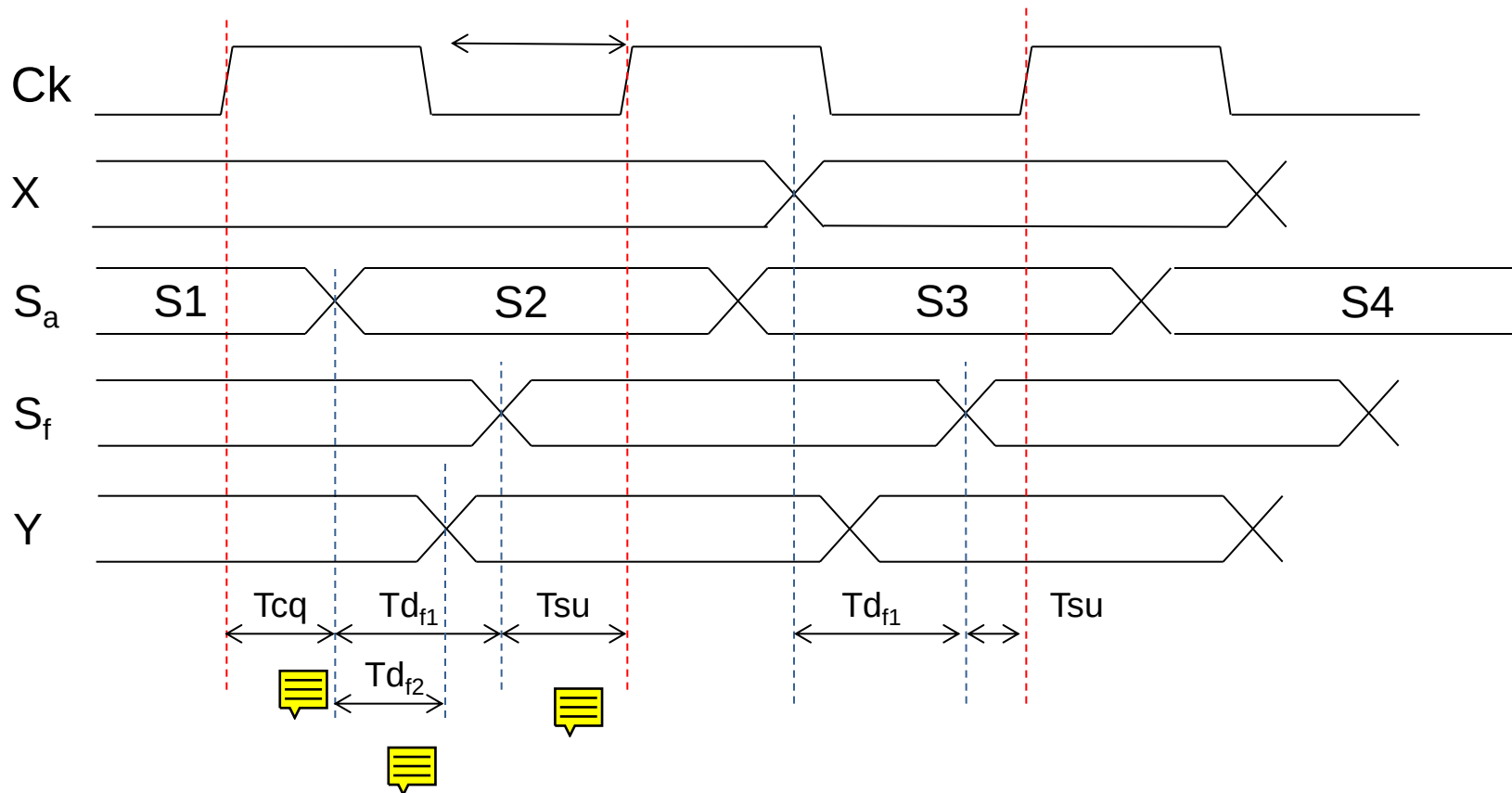
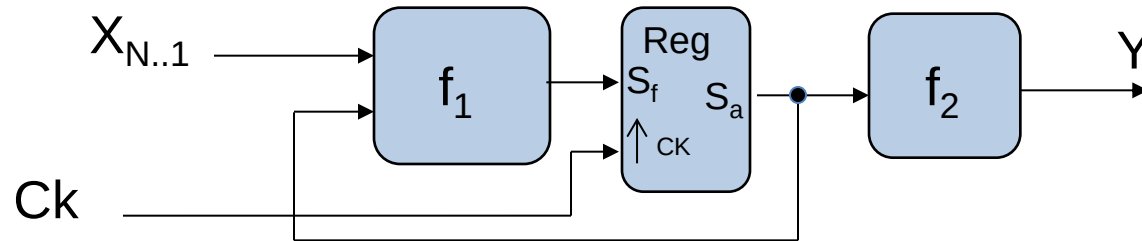
$$Q_0 = \overline{R} \overline{I_0}$$

$$Q_1 = \overline{R} \overline{I_1} I_0 + \overline{R} I_1 \overline{I_0}$$

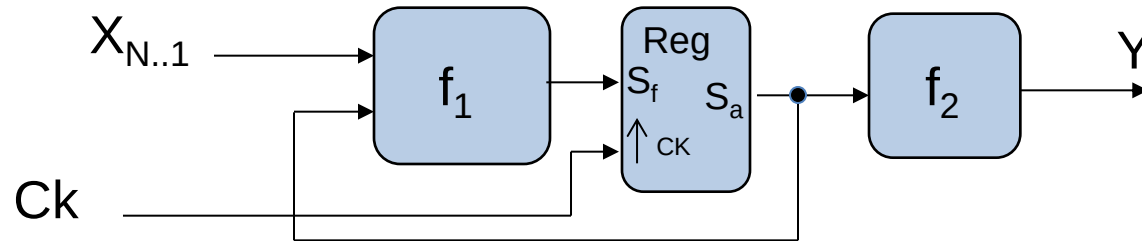
$$Q_2 = R \overline{I_2} I_1 I_0 + \overline{R} I_2 \overline{I_1} \overline{I_0} + \overline{R} I_2 \overline{I_1} I_0 + \overline{R} I_2 I_1 \overline{I_0}$$

➤  $f_2$  la funzione identità

## - Temporizzazione Reti Sequenziali Sincrone -



## - Temporizzazione Reti Sequenziali Sincrone -



$$f_{\max} = 1/T_{\min}$$

$$T_{\min} = T_{cq} + T_{d_{f1}} + T_{su}$$

$$T_{cq} = T_{cq} + T_{d_{f2}} \quad \text{Ritardo fra clock e uscita Y}$$

$$T_{sux} = T_{d_{f1}} + T_{su} \quad \text{Set-up degli ingressi X}$$

Nota: nell'implementazione in CPLD/FPGA è necessario considerare i ritardi delle connessioni



# Laboratorio di Elettronica Digitale

## - Introduzione al Linguaggio VHDL -

Prof. Stefano Ricci



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

**DINFO**

DIPARTIMENTO DI  
INGEGNERIA  
DELL'INFORMAZIONE

Updated 02/2025

# ATTRIBUTO 'EVENT

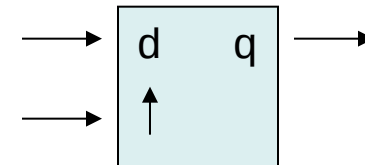
- Al fine di poter controllare l'evoluzione dei segnali è possibile associare ai segnali degli attributi.
- Ve ne sono vari, tratteremo solo 'event
- Un segnale genera un event ogni qualvolta cambia di valore
- Per esempio su di un segnale di tipo bit si verifica un event ogni volta che ha una transizione da 0 a 1 o viceversa.
- un vettore genera un event se un suo componente lo genera

La variabile booleana s'event assume valore true ogni volta che s genera un evento

Si noti l'apostrofo

# ATTRIBUTO 'EVENT

- 'event è utile per individuare le transizioni di clock
- in particolare clock'event and clock = '1' sarà 'true' solo sul fronte di salita del clock
- Esempio: flip flop d attivo su fronte di salita



```
entity ff_ent is
  port ( d, ck: in bit;
        q: out bit);
end ff_ent;
architecture ff_arc of ff_ent is
  begin
    ff: process ( ck )
    begin
      if (ck'event and ck = '1') then
        q <= d;
      end if;
    end process ff;
  end ff_arc ;
```

Nota: d non è nella sensitivity list!

variazione di d non va considerata! => macchina sincrona

il costrutto if... seleziona solo i fronti di salita del segnale ck

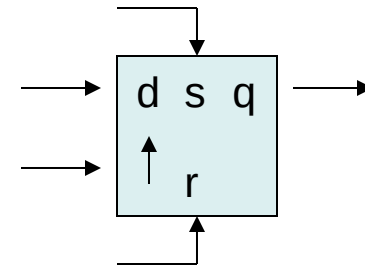
evento sul fronte di salita  
=> utile per variazione degli stati per macchina sequenziale sincrona

# ATTRIBUTO 'EVENT

set/reset avviene immediatamente  
=> non aspetta fronte di salita  
=> devono stare in sensitivity list.

➤ Esempio: flip flop d attivo su fronte di salita con **set – reset asincroni**

```
entity ff_ent is
  port ( d, s, r, ck: in bit;
        q: out bit);
end ff_ent;
architecture ff_arc of ff_ent is
  begin
    ff: process ( s,r,ck )
      begin
        if s='1' then
          q <= '1';
        elsif r = '1' then
          q <= '0';
        elsif (ck'event and ck = '1') then
          q <= d;
        end if;
      end process ff;
    end ff_arc ;
```



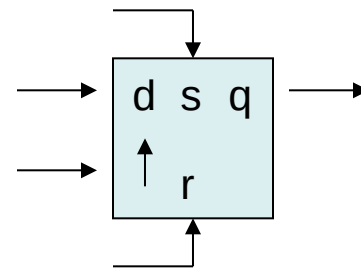
In questo esempio s ha precedenza su r. Infatti se s = r = '1' il ff va a '1'. La *sequenza* delle istruzioni ha importanza. Si conferma che *if-then-else* è un costrutto *sequenziale*.



# ATTRIBUTO 'EVENT

➤ Esempio: flip flop d attivo su fronte di salita con **set – reset sincroni**

```
entity ff_ent is
  port ( d, s, r, ck: in bit;
        q: out bit);
end ff_ent;
architecture ff_arc of ff_ent is
  begin
    ff: process ( ck )
      begin
        if (ck'event and ck = '1') then
          tutto deve accadere solo sul fronte di clk
          if r = '1' then
            q <= '0';
          elsif s = '1' then
            q <= '1';
          else
            q <= d;
          end if;
        end if;
      end process ff;
end ff_arc ;
```



In questo esempio r ha precedenza su s.

# CIRCUITI COMBINATORI E SEQUENZIALI

- Utilizzando le strutture del VHDL in teoria è possibile scrivere codici che realizzano funzioni molto 'strane'
- Anche limitandosi a usare istruzioni sintetizzabili (per esempio non usando 'wait' o 'real') non è detto che tutto ciò che si scrive sia sintetizzabile.
- Non è sufficiente seguire le regole sintattiche che abbiamo visto per scrivere codice che possa avere una corrispondenza hardware
- Inoltre le regole della "buona programmazione" impongono dei limiti ancora più ristretti su come effettivamente usare il codice VHDL

In pratica vi sono, al di là della sintassi VHDL, delle strutture e regole più o meno assodate per realizzare circuiti combinatori e sequenziali.

# BLOCCHI SEQUENZIALI: REGOLE GENERALI

- Usare SOLO strutture sequenziali sincrone
- TUTTE le uscite sono aggiornate sul fronte di salita del segnale di clock
- Unica eccezione: può esserci reset (o set) asincrono
- Evitare assolutamente di utilizzare elementi di memoria asincroni (latch)

➤ In questo modo la logica sintetizzata si mappa facilmente con l'architettura della CPLD/FPGA (che ha in HW ff D)

Inoltre l'ambiente di sviluppo può calcolare facilmente le prestazioni e guidare il fitting del dispositivo

# BLOCCHI SEQUENZIALI: REGOLE GENERALI

```
entity sec_ent is
  port ( .... : in bit;
        .... : out bit);
end sec_ent ;
architecture sync_arc of sec_ent is
  begin
    sync: process (reset, ck )
      begin
        if reset='1' then
          reset di tutte le uscite..
        elsif (ck'event and ck = '1') then
          logica sincrona...
        end if;
      end process sync;
    end sync_arc ;
```

Il processo è sensibile solo al clock ed (eventualmente) il reset (set)

se c'è reset ha priorità => sta fuori l'if  
=> guarda prima il reset  
=> il reset è l'unica eccezione! (altrimenti  
~~sta tutto dentro ck' event~~)

In sensitivity list deve essere presente solo il clock e eventualmente il reset (set)



# BLOCCHI SEQUENZIALI: ESEMPIO

➤ Esempio: contatore up/down 4 bit con load sincrono e reset asincrono

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_signed.all;

entity cont_ent is
  port (
    up_down, reset, load, ck : in std_logic;
    data : in std_logic_vector (3 downto 0);
    cnt : out std_logic_vector (3 downto 0);
  end cont_ent ;    occhio cnt è un segnale di uscita...

  architecture arc_arc of cont_ent is
  begin
    sync: process (reset, ck )
      --
    end process sync;
  end arc_arc ;
```

```
sync: process (reset, ck )
begin
  if reset='1' then
    cnt <= "0000";
  elsif (ck'event and ck = '1') then
    if load = '1' then    segnale di
                          caricamento
      cnt <= data;
    elsif up_down = '1' then
      cnt <= cnt + "0001";
    else
      cnt <= cnt - "0001";
    end if;
  end if;
end process sync;
```

Vedi note.....

# BLOCCHI SEQUENZIALI: NOTE

- (1) Le librerie inserite consentono l'uso dei `std_logic` e contengono la definizione dell'operatori '+' e '-' sui `std_logic` (`ieee.std_logic_signed`).  
Con questa libreria i vettori sono considerati valori interi con rappresentazione complemento a 2 con segno
- Il controllo sintattico del codice fornisce comunque un errore: `cnt` è una porta con direzione `out` e pertanto non può essere letto ma solo scritto.  
L'istruzione (2) comporta invece una lettura di `cnt`.

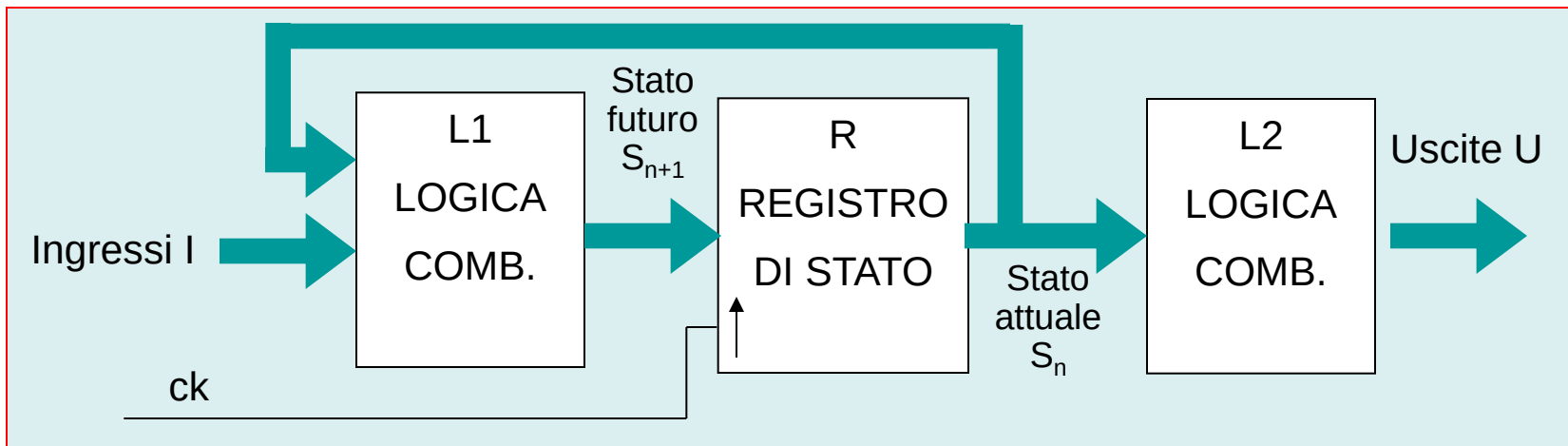
- Le porte **in** possono essere solo LETTE, le porte **out** solo SCRITTE
- i signal si leggono e scrivono

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.std_logic_signed.all;  
  
entity cont_ent is  
  port ( ck : in std_logic;  
        cnt : out std_logic_vector (3 downto 0) );  
end cont_ent ;  
  
architecture arc of cont_ent is  
  signal cnt_int std_logic_vector (3 downto 0);  
begin  
  cnt <= cnt_int;  
  sync: process (ck )  
  begin  
    if (ck'event and ck = '1') then  
      cnt_int <= cnt_int + "0001";  
    end if;  
  end process sync;  
end arc_arc;
```

Risolvero il problema definendo un segnale 'cnt\_int', che può essere sia scritto che letto, e assegnando quindi cnt\_int su cnt, che così viene solo scritto

# MACCHINE A STATI IN VHDL

## ➤ Macchina di Moore



L1 Gestisce i cambi di stato

$$S_{n+1} = f_s(S_n, I)$$

L2 Gestisce le uscite

$$U = f_u(S_n)$$

➤ L1 + R => Processo sincrono

➤ L2 => Processo combinatorio



# MACCHINA DI MOORE

```
library ieee;
use ieee.std_logic_1164.all;

entity sm_ent is
  port ( reset, ck : in std_logic;
        I : in std_logic_vector (xx downto 0);
        U: out std_logic_vector (xx downto 0));
end sm_ent ;

architecture arc_arc of sm_ent is
  signal stato_a, stato_f: std_logic_vector (xx downto 0)
begin
  L1: process (stato_a, I)
    stato_f <= ...
  end process L1;

  R: process (reset, ck)
    stato_a <= ...      stato-futuro copiato ad ogni fronte di salita
  end process R;

  L2: process (stato_a)
    U <= ...
  end process L1;
end arc_arc ;
```

- Si definiscono il signal per lo stato attuale (stato\_a) e futuro (stato\_f)
- Il processo combinatorio L1 determina stato\_f a partire da stato\_a e gli ingressi I
- Il processo sincrono R rappresenta il registro di stato che sul fronte di clock aggiorna stato\_a a stato\_f
- Il processo combinatorio L2 determina le uscite U a partire da stato\_a

# MACCHINA DI MOORE

```
L1: process (stato_a, l)
begin
  case stato_a is
    when "0..0" =>
      if l = .... then
        stato_f <= ....;
      elsif l = .... then
        stato_f <= ....;
      else
        stato_f <= ....;
      end if;
    when "0..1" =>
      if l = .... then
        stato_f <= ....;
      elsif l = .... then
        stato_f <= ....;
      else
        stato_f <= ....;
      end if;
    when "1..0" =>
      ...
    when others =>
      stato_f <= "00...00";
  end case;
end process L1;
```

in base ad ingressi  
decido stato futuro

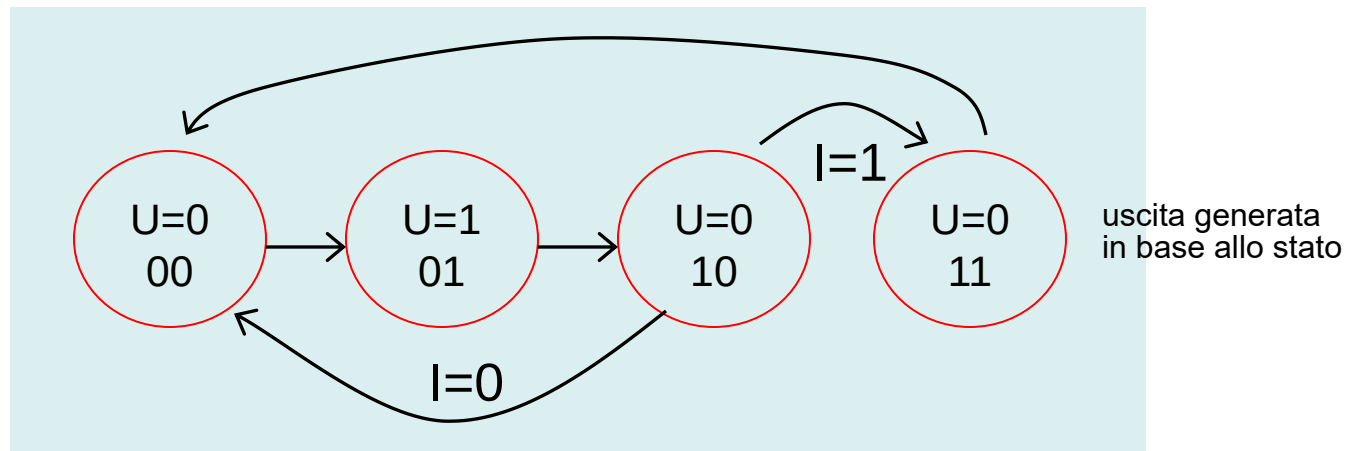
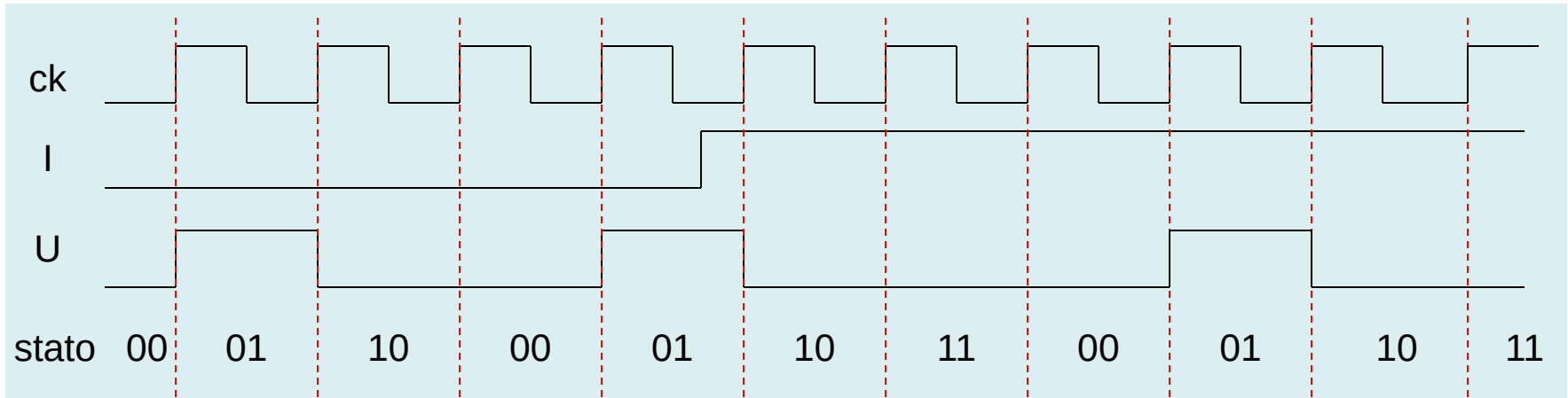
```
L2: process (stato_a)
begin
  case stato_a is
    when "0..0" =>
      U <= "00..10";
    when "0..1" =>
      U <= "00..10";
    ...
    when "1..0" =>
      U <= "00..10";
    when others =>
      U <= "00...00";
  end case;
end process L2;
```

qui decido  
il vettore delle uscite  
(spesso non presente)

```
R: process (reset, ck )
begin
  if reset='1' then
    stato_a <= "00...00";
  elsif (ck'event and ck = '1') then
    stato_a <= stato_f;
  end if;
end process R;
```

# MACCHINA DI MOORE

➤ Esempio: macchina che, a seconda di un ingresso genera un impulso ogni 3 clk piuttosto che ogni 4



# MACCHINA DI MOORE

## ➤ Esempio: codifica in VHDL

```
L1: process (stato_a, l)
begin
    case stato_a is
        when "00" =>
            stato_f <= "01";
        when "01" =>
            stato_f <= "10";
        when "10" =>
            if l = '0' then
                stato_f <= "00";
            else
                stato_f <= "11";
            end if;
        when "11" =>
            stato_f <= "00";
        when others =>
            stato_f <= "00";
    end case;
end process L1;
```

```
entity ck_div is
    port ( reset, ck, l : in std_logic;
          u: out std_logic );
end ck_div;
```

```
L2: process (stato_a)
begin
    case stato_a is
        when "01" =>
            U <= '1';
        when others =>
            U <= '0';
    end case;
end process L2;
```

```
R: process (reset, ck )
begin
    if reset='1' then
        stato_a <= "00";
    elsif (ck'event and ck = '1') then
        stato_a <= stato_f;
    end if;
end process R;
```

# MACCHINA DI MOORE

```
signal stato: std_logic_vector (1 downto 0)
begin
L1R: process (reset, ck )
begin
    if reset='1' then
        stato <= "00";
    elsif (ck'event and ck = '1') then
        case stato is
            when "00" =>
                stato <= "01";
            when "01" =>
                stato <= "10";
            when "10" =>
                if l = '0' then
                    stato <= "00";
                else
                    stato <= "11";
                end if;
            when "11" =>
                stato <= "00";
        end case;
    end if;
end process L1R;
```

➤ Si possono integrare i processi L1 e R utilizzando il solo signal 'stato' al posto di 'stato\_a' e 'stato\_f'

# MACCHINA DI MOORE

```
type state_type is (s0, s1, s2, s3);  
signal stato: state_type;  
begin  
  L1R: process (reset, ck )  
  begin  
    if reset='1' then  
      stato <= s0;  
    elsif (ck'event and ck = '1') then  
      case stato is  
        when s0 =>  
          stato <= s1;  
        when s1 =>  
          stato <= s2;  
        when s2 =>  
          if l = '0' then  
            stato <= s0;  
          else  
            stato <= s3;  
          end if;  
        when s3 =>  
          stato <= s0;  
        end case;  
      end if;  
    end process L1R;
```

➤ Si può definire un nuovo 'type' con il costrutto **type xx is (...)**, quindi istanziare un signal del tipo definito.

➤ In questo modo non specifico i valori binari di 'stato' lasciando l'ottimizzazione al compilatore



# MACCHINA DI MOORE

```
L1L2R: process (reset, ck )
begin
    if reset='1' then
        stato <= s0;
    elsif (ck'event and ck = '1') then
        case stato is
            when s0 =>
                stato <= s1;
                U <= '1';
            when s1 =>
                stato <= s2;
                U <= '0';
            when s2 =>
                U <= '0';
                if l = '0' then
                    stato <= s0;
                else
                    stato <= s3;
                end if;
            when s3 =>
                U <= '0';
                stato <= s0;
            end case;
        end if;
    end process L1L2R;
```

➤ E' possibile codificare la macchina in un unico processo che specifichi anche l'uscita.

➤ **Attenzione: in questo caso l'uscita va "pre-decodificata",** infatti è schedulata a 1 durante lo stato s0, contemporaneamente alla transizione s0->s1 

➤ questo consente di diminuire il Tckq di U, ma dipende molto dal compilatore

# MACCHINA DI MOORE

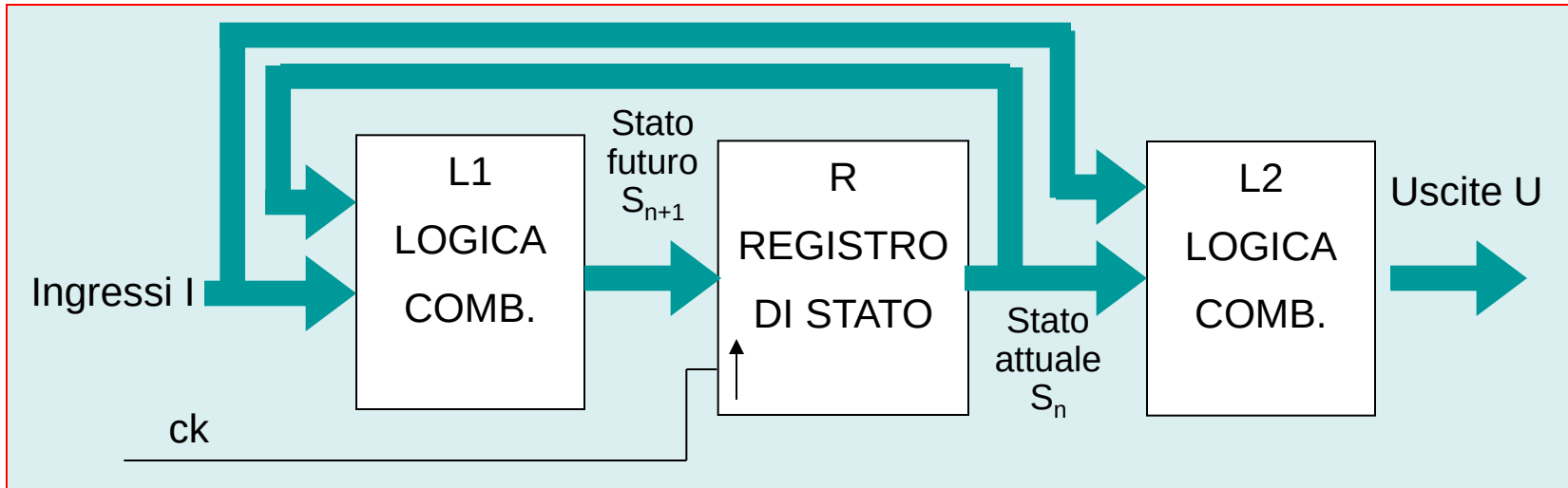
```
L1L2R: process ( reset, ck )  
begin  
    if reset='1' then  
        stato <= s0;  
    elsif (ck'event and ck = '1') then  
        U <= '0';  
        case stato is  
            when s0 =>  
                stato <= s1;  
                U <= '1';  
            when s1 =>  
                stato <= s2;  
            when s2 =>  
                if l = '0' then  
                    stato <= s0;  
                else  
                    stato <= s3;  
                end if;  
            when s3 =>  
                stato <= s0;  
            end case;  
        end if;  
    end process L1L2R;
```

- Equivalente alla precedente, ma il codice è più compatto
- Sfruttando la sequenzialità delle istruzioni, l'assegnazione  $U \leq '0'$  vale come default fintanto che non venga sovrascritta dalla successiva



# MACCHINE A STATI IN VHDL

## ➤ Macchina di Mealy



L1 Gestisce i cambi di stato

$$S_{n+1} = f_s(S_n, I)$$

L2 Gestisce le uscite

$$U = f_u(S_n, I)$$

➤ R1 => Processo sincrono

➤ L1 + L2 => Processo combinatorio

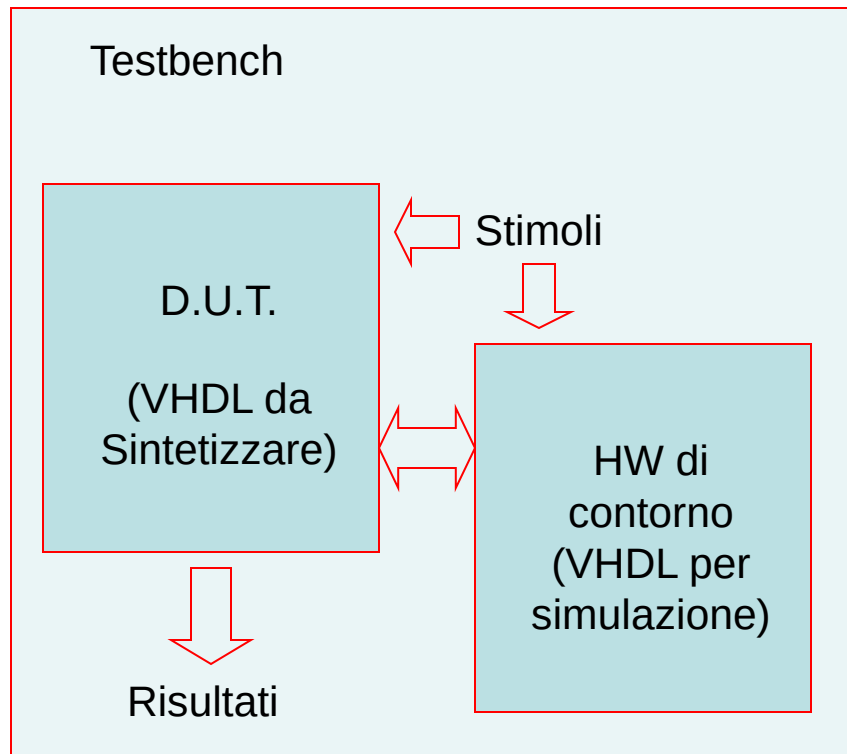
# MACCHINA DI MEALY

```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity sm_ent is  
  port ( reset, ck : in std_logic;  
        I : in std_logic_vector (xx downto 0);  
        U: out std_logic_vector (xx downto 0));  
end sm_ent ;  
  
architecture arc_arc of sm_ent is  
  type state_type is (s0, s1, s2, s3,...);  
  signal stato: state_type;  
  begin  
    L1R: process (reset, ck)  
      stato <= ...  
    end process L1R;  
  
    L2: process (stato, I)  
      U <= ...  
    end process L2;  
end arc_arc ;
```

➤ Si codifica in modo simile alla macchina di Moore, cambia solo il processo L2 che è sensibile anche a I

# TESTBENCH

➤ Il codice VHDL destinato alla sintesi viene testato con simulazioni effettuate creando un ambiente chiamato “testbench”



- Nel testbench si inseriscono altre componenti che simulano eventuale altro HW necessario al test
- Quindi il testbench genera gli stimoli e raccoglie i risultati che poi vengono analizzati
- I tool di simulazione (per es. Modelsim di Mentor) consentono di visualizzare non solo i risultati ma anche tutti i segnali interni

# TESTBENCH

```
library ieee;  
use ieee.std_logic_1164.all;  
use std.textio.all;  
...  
entity tstbench is  
end tstbench ;  
  
architecture arc of tstbench is  
    signal...  
    ...  
    component dut port(...);  
    end component ;  
    ...  
    component hw1 port(...);  
    end component ;  
begin  
    p1: process (...)  
    end process p1;  
    ...  
    pn: process (...)  
    end process pn;  
    d1:dut port map (...);  
    d2:hw1 port map (...);  
end arc;
```

- Si includono varie librerie, fra cui quelle contenenti comandi per l'accesso ai file
- Il testbench ha un'interfaccia "chiusa"
- Si dichiarano e istanziano il componente da testare e gli eventuali dispositivi di contorno
- Con i vari processi si generano gli stimoli e si raccolgono i segnali di interesse
- Poiché il testbench non deve essere sintetizzato, si possono usare tutte le strutture, compreso, per esempio, l'accesso ai file per leggere gli stimoli e salvare i risultati

# TESTBENCH

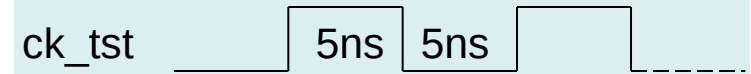
```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity tstbench is  
end tstbench ;  
  
architecture arc of tstbench is  
    signal reset_tst, ck_tst, l_tst, u_tst: std_logic  
    component ck_div  
        port ( reset, ck, l : in std_logic;  
              u: out std_logic );  
    end component ;  
  
begin  
    ckp: process (...)  
    end process ckp;  
    resetp: process (...)  
    end process resetp;  
    lp: process (...)  
    end process lp;  
    d1: ck_div port map (  
        reset_tst, ck_tst, l_tst, u_tst);  
end arc;
```

Esempio: testbench di ck\_div

- Si dichiara componente sotto test
- Si definisce un segnale \*\_tst per ogni segnale in interfaccia
- Si usa un processo per ciascun stimolo (ingresso) da generare
- Si istanzia il componente

# TESTBENCH

```
ckp: process
begin
    ck_tst <= '0';
    wait for 5 ns;
    ck_tst <= '1';
    wait for 5 ns;
end process ckp;
```



$$T = 10 \text{ ns} \quad \Leftrightarrow \quad f = 100 \text{ MHz}$$

- le istruzioni **wait** sono mutualmente esclusive rispetto alla **sensitivity list**
- ❖ la sensitivity list equivale ad un wait for “eventi sui segnali elencati”
- ❖ ad ogni sospensione le assegnazioni schedulate vengono eseguite
- ❖ **wait for time** sospende il processo per *time*

# TESTBENCH

```
resetp: process
begin
    reset_tst <= '1';
    wait for 10 ns;
    reset_tst <= '0';
    wait;
end process resetp;
```

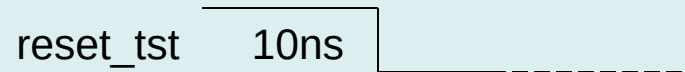


Diagram showing the signal reset\_tst. It starts at a high level, then drops to a low level after a 10ns delay, and remains low. The signal is represented by a horizontal line at the high level, followed by a vertical drop, then a horizontal line at the low level. The 10ns delay is indicated by a horizontal line segment above the drop.

```
lp: process
begin
    l_tst <= '0';
    wait for 35 ns;
    l_tst <= '1';
    wait;
end process lp;
```

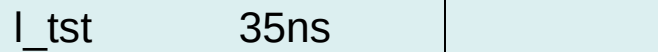
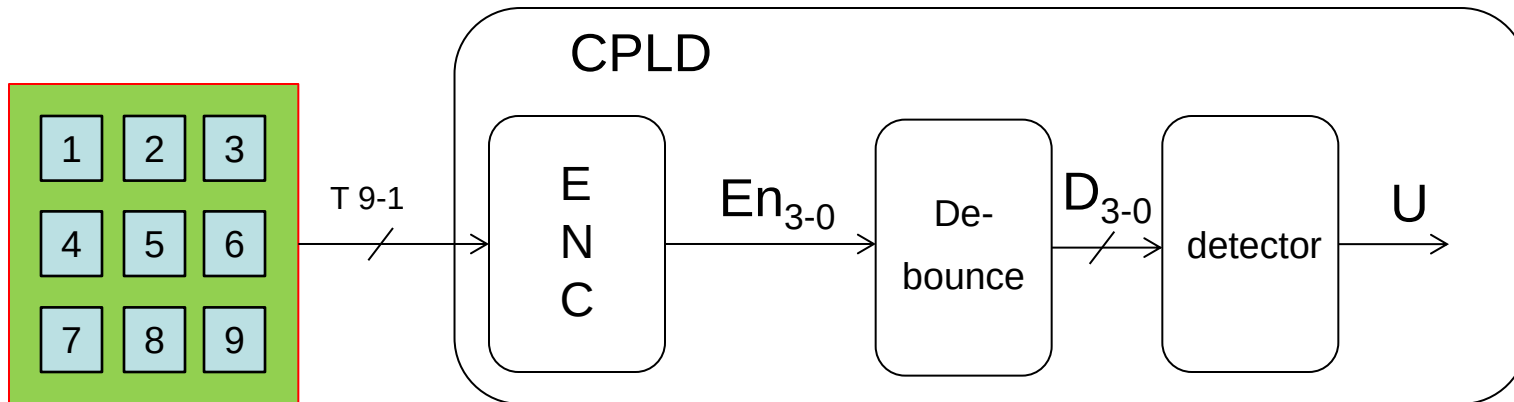


Diagram showing the signal l\_tst. It starts at a low level, then rises to a high level after a 35ns delay, and remains high. The signal is represented by a horizontal line at the low level, followed by a vertical rise, then a horizontal line at the high level. The 35ns delay is indicated by a horizontal line segment above the rise.

➤ **wait** (senza **for**...) sospende il processo indefinitamente

## EXAMPLE: CODE DETECTION



Keyboard output:

| key  | T9 | T8 | T7 | T6 | T5 | T4 | T3 | T2 | T1 |
|------|----|----|----|----|----|----|----|----|----|
| none | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 1    | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  |
| 2    | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  |
| 3    | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  |
| 4    | 0  | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  |
| 5    | 0  | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  |
| 6    | 0  | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  |
| 7    | 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  |
| 8    | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 9    | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |

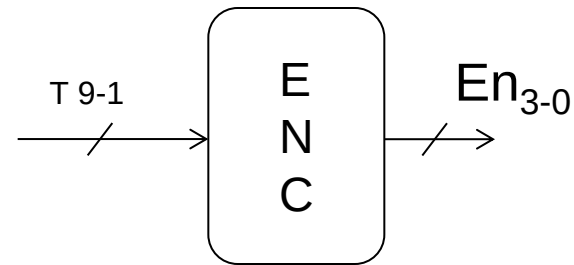


## EXAMPLE: CODE DETECTION

### ENCODER

#### Blocco combinatorio

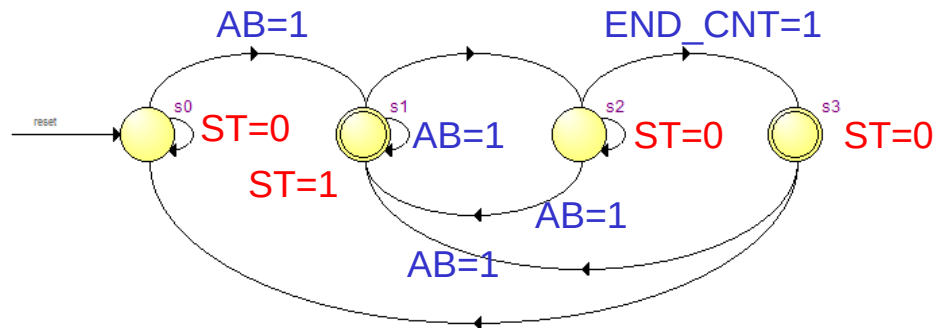
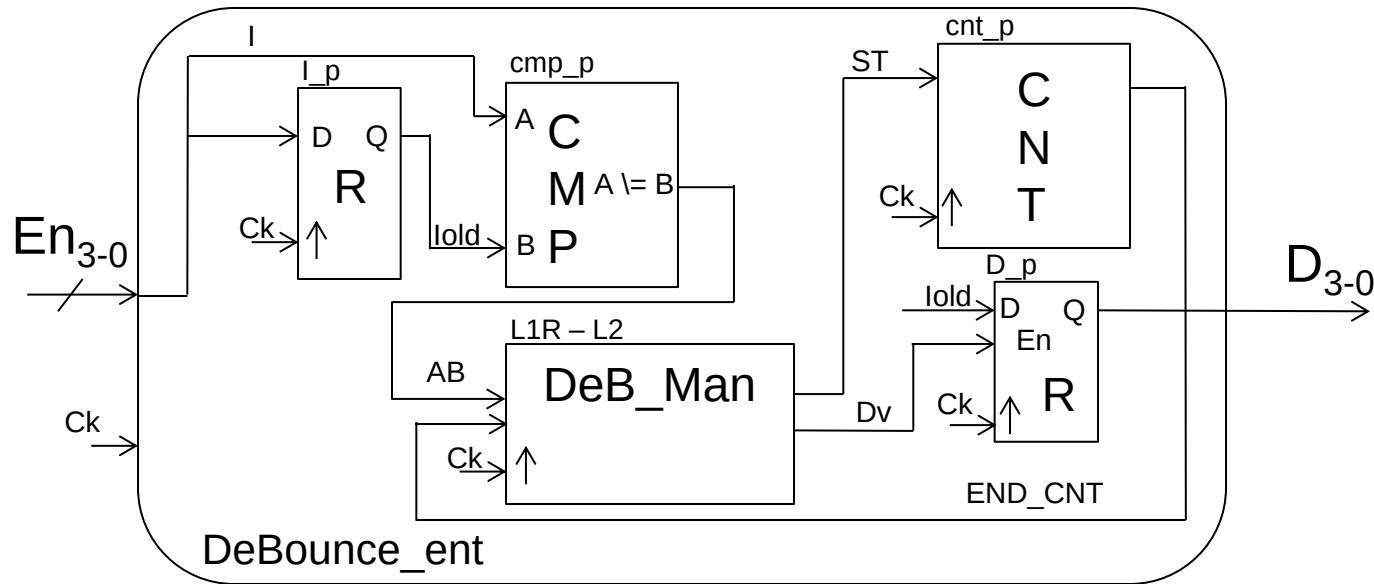
```
entity Penc_ent is
  port (   T: in bit_vector(9 downto 1);
          En: out bit_vector(3 downto 0) );
end Penc_ent;
architecture Penc_arc of Penc_ent is
  begin
    Penc: process (T)
    begin
      if      T(1) = '1' then En <= "0001";
      elsif  T(2) = '1' then En <= "0010";
      elsif  T(3) = '1' then En <= "0011";
      elsif  T(4) = '1' then En <= "0100";
      elsif  T(5) = '1' then En <= "0101";
      elsif  T(6) = '1' then En <= "0110";
      elsif  T(7) = '1' then En <= "0111";
      elsif  T(8) = '1' then En <= "1000";
      elsif  T(9) = '1' then En <= "1001";
      else
        En <= "0000";
      end if;
    end process Penc;
  end Penc_arc ;
```



## EXAMPLE: CODE DETECTION

De-Bounce

Blocco misto combinatorio / sequenziale



State transitions of  
DeB\_Man

## EXAMPLE: CODE DETECTION

Detector

